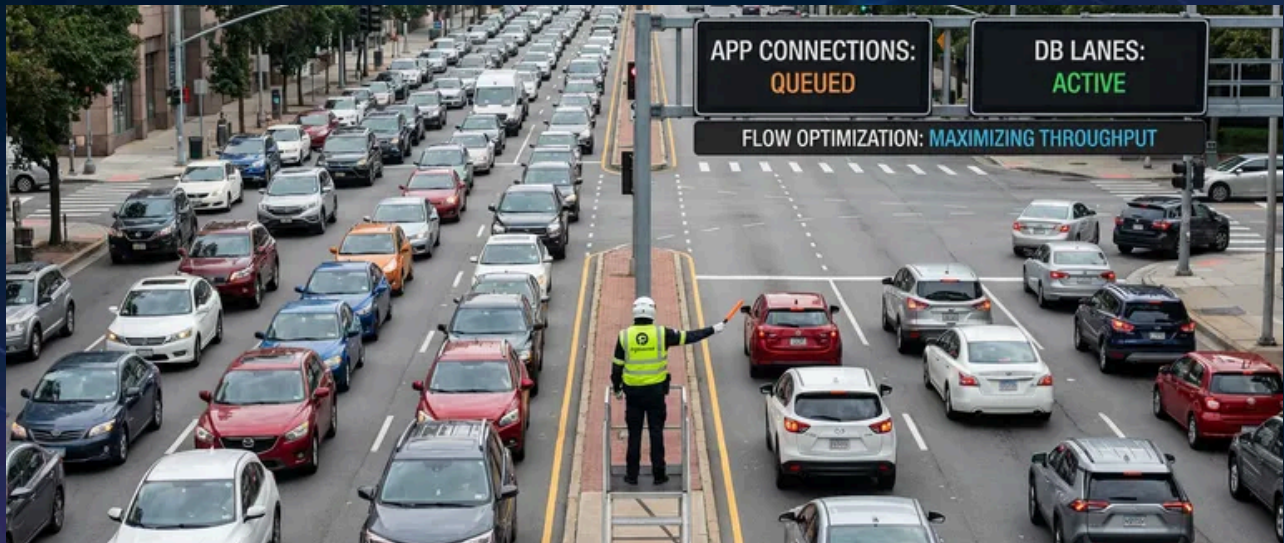


PostgreSQL Connections: Pooling and Saturation



Published on November 5, 2023



Webstack
Builders

Table of Contents

Connection Fundamentals	3
How PostgreSQL Handles Connections	3
Connection Costs	5
Connection Pooling	7
Pooling Architectures	7
PgBouncer Configuration	9
Pool Sizing	10
Sizing Calculations	10
Application Pool Configuration	12
Saturation Signals	15
Metrics to Monitor	15
Monitoring Dashboard	17
Failure Patterns	18
Connection Exhaustion Cascade	18
Recovery Strategies	20
Conclusion	22
Reference Queries	23
Connection Snapshot	24
Connection Age Distribution	24
Lock Blocking Analysis	25
Kill Stuck Sessions	25



Every PostgreSQL connection costs memory – roughly 5-10MB per connection – and requires a dedicated backend process. A database configured for 200 max connections isn't "safe at 199." It's already under strain well before hitting that limit. Connection management isn't about avoiding hard limits; it's about understanding the soft degradation that happens as connections accumulate. Connection exhaustion doesn't announce itself with a clean error. It manifests as slow queries, timeouts, and cascading failures across your application tier.

I've seen this pattern repeatedly: a team runs PostgreSQL with `max_connections=200`, comfortable at typical usage of 50. Traffic spikes, instances autoscale from 5 to 20, each with a pool of 20. Four hundred connections hit a 200 limit. Rejections cascade – threads block, health checks fail, more scaling triggers more connection requests.

⚠ WARNING

The most common PostgreSQL scaling mistake: configuring `max_connections` high "just in case." Each connection costs memory and CPU. A database with 1000 `max_connections` will perform worse than one with 200 – even at the same actual connection count – because of reservation overhead.

After adding PgBouncer (PostgreSQL Connection Pooler), properly sizing pools, and implementing connection backpressure, the next traffic spike looks different. Database connections stay at 200. The application queues requests appropriately. Latency increases, but nothing fails. The lesson: connection limits aren't about capacity – they're about graceful degradation.

Connection Fundamentals

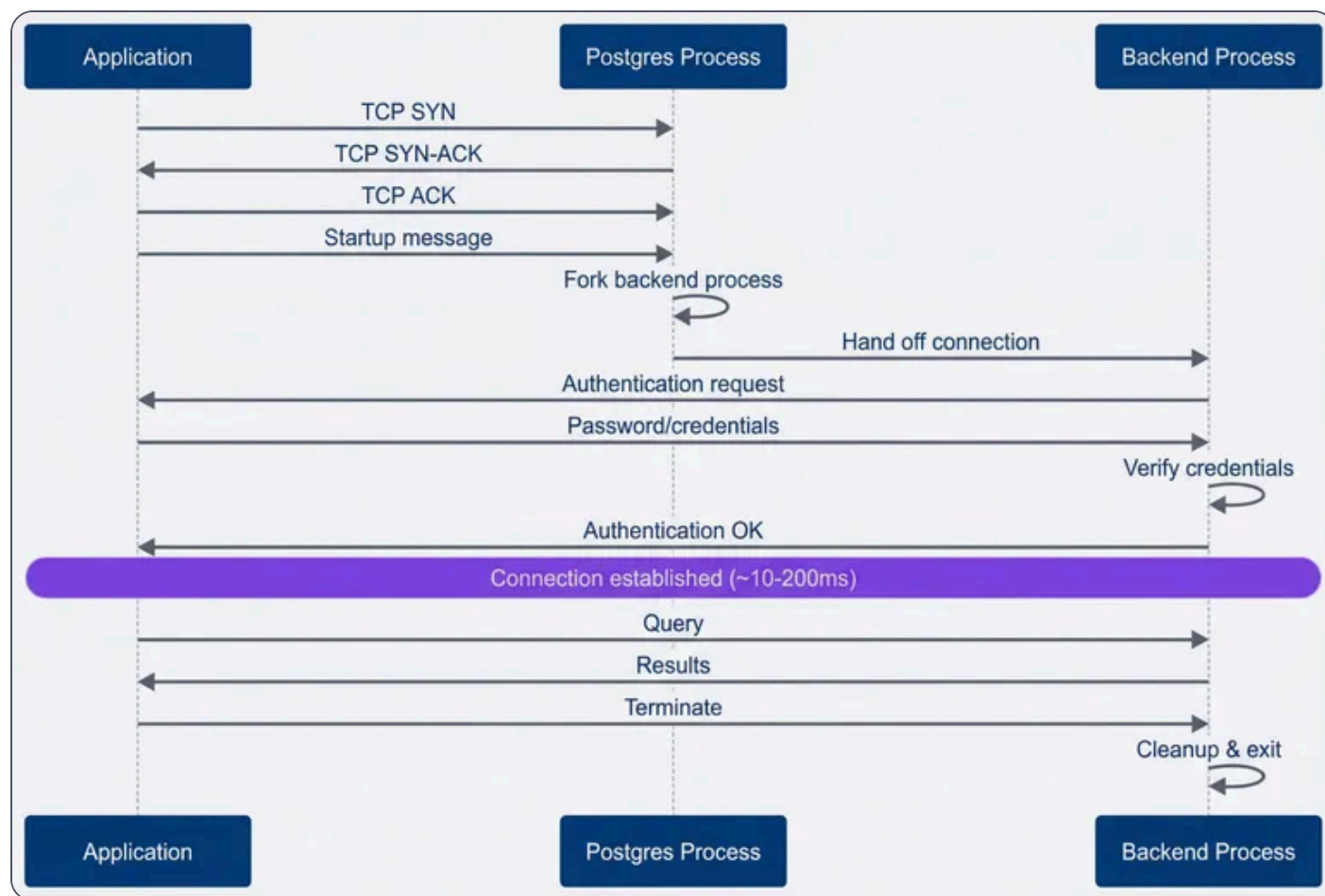
How PostgreSQL Handles Connections

PostgreSQL uses a process-per-connection model. When a client connects, the main **postgres process** (the server daemon) forks a new **backend process** dedicated to that client. Each backend handles all queries for its **connection** (the TCP (Transmission Control Protocol) session between client and backend) until termination. In `pg_stat_activity`, these appear as rows with `backend_type = 'client backend'`. The fork-based model provides strong isolation – a crash in one backend doesn't affect others – but it comes with overhead.



Establishing a new connection involves multiple steps: TCP (Transmission Control Protocol) handshake (1ms locally, 10-100ms over network), process fork (1-5ms), authentication (1-10ms depending on method), and TLS (Transport Layer Security) handshake if configured (5-50ms). Total: 10-200ms for a new connection. For web applications making dozens of queries per request, creating a fresh connection each time is prohibitively expensive.

Each backend process consumes resources independent of whether it's actively running queries. The base memory footprint is roughly 5-10MB per connection, plus `work_mem` allocation when executing operations (4MB default, but can be much higher for complex queries). CPU overhead comes from OS scheduler contention – more backend processes means more context switching as the kernel cycles between them, even when most are idle.



Connection establishment sequence.



The `max_connections` setting determines how many backend processes can exist simultaneously. The conventional wisdom is to set it high “for safety,” but this backfires. A database configured for 1000 connections reserves memory for 1000 potential backends even when only 50 are connected. The rule of thumb for bare metal: `max_connections = (CPU cores × 2) + effective_spindle_count`¹. For SSDs, `CPU cores × 4` works, but staying conservative (50-100) with a connection pooler is usually better.

max_connections	Use Case	Memory Overhead	Notes
50-100	Small deployments with pooler	~500MB-1GB	Recommended baseline
100-300	Medium deployments	~1-3GB	Watch for contention
500-1000	Large deployments	~5-10GB	Usually wrong; use pooler instead

max_connections settings and their implications.

Connection Costs

Understanding where your connections come from – and what state they’re in – is critical for diagnosing saturation. The `pg_stat_activity` view shows all current connections with their state, user, application, and current query.

connection-monitoring.sql

```

1  -- Current connection count by state
2  SELECT
3      state,
4      count(*) as connections,
5      round(100.0 * count(*) / sum(count(*) over(), 1) as percentage
6  FROM pg_stat_activity
7  WHERE backend_type = 'client backend'
8  GROUP BY state
9  ORDER BY connections DESC;
10
11 -- Connection count by application/user
12 SELECT
13     username,
14     application_name,
```



```

15     client_addr,
16     count(*) as connections,
17     count(*) FILTER (WHERE state = 'active') as active,
18     count(*) FILTER (WHERE state = 'idle') as idle,
19     count(*) FILTER (WHERE state = 'idle in transaction') as idle_in_txn
20 FROM pg_stat_activity
21 WHERE backend_type = 'client backend'
22 GROUP BY username, application_name, client_addr
23 ORDER BY connections DESC;
24
25 -- Memory usage estimate (rough)
26 SELECT
27     count(*) as total_connections,
28     count(*) * 10 as estimated_memory_mb,
29     current_setting('max_connections')::int as max_connections,
30     round(100.0 * count(*) / current_setting('max_connections')::int, 1) as
utilization_pct
31 FROM pg_stat_activity
32 WHERE backend_type = 'client backend';

```

Connection monitoring queries for PostgreSQL.

Connection state tells you what each backend is doing. Most connections in a healthy pooled application should be **idle** –waiting for the next query. **Active** connections are executing queries; a high percentage of active connections indicates either long-running queries or genuine load.

#	Connection State	Description	Concern Level
1	active	Executing query	Normal if short
2	idle	Waiting for query	Normal (pooled)
3	idle in transaction	Transaction open, no query	High (blocks vacuum)
4	idle in transaction (aborted)	Failed transaction, not rolled back	Critical

Connection states and their implications.



“Idle in transaction” connections deserve special attention: they hold locks, prevent vacuum from cleaning up dead tuples, and consume connection slots. Set `idle_in_transaction_session_timeout` to kill them automatically.

Connection Pooling

Pooling Architectures

Connection pooling comes in two flavors: application-side pools and external poolers. Both solve the same problem – avoiding the overhead of creating new connections – but they scale differently.

Application-side pools

Live within your application process. HikariCP (Java), node-postgres pool, SQLAlchemy (Python), and Go's database/sql all provide built-in pooling. The advantage is simplicity: no additional infrastructure, per-application configuration, and full support for prepared statements. The disadvantage is that scaling your application scales your connection count. Five application instances with pools of 20 connections each means 100 database connections. Twenty instances means 400. This couples your horizontal scaling to your database connection limits.



External poolers

Sit between applications and the database as dedicated services. PgBouncer, Pgpool-II, and AWS RDS Proxy all work this way. The key benefit is connection multiplexing – many application connections can share fewer database connections. Your application can scale to 100 instances while the database sees a fixed 50 connections. The tradeoff is additional infrastructure and a potential single point of failure (mitigated by running multiple pooler instances).



Managed database services

Often provide built-in pooling. AWS RDS Proxy, Azure Database for PostgreSQL Flexible Server (with PgBouncer), and Google Cloud SQL don't offer native pooling but work well with Cloud SQL Auth Proxy for connection management. If you're on a managed platform, check whether pooling is available before deploying your own PgBouncer – the managed option is usually simpler to operate.



Approach	Examples	Best For	Connection Scaling
Application-side	HikariCP, pg-pool, SQLAlchemy	Single app, prepared statements	instances × pool_size
External pooler	PgBouncer, RDS Proxy	Multi-app, auto-scaling	Fixed, independent of apps

Pooling approaches compared.

External poolers support different **pooling modes** that determine when connections are returned to the pool:

Session mode

Client gets a dedicated server connection for its entire session. No multiplexing benefit, but full prepared statement support. Use for long-running sessions or heavy prepared statement workloads.



Transaction mode

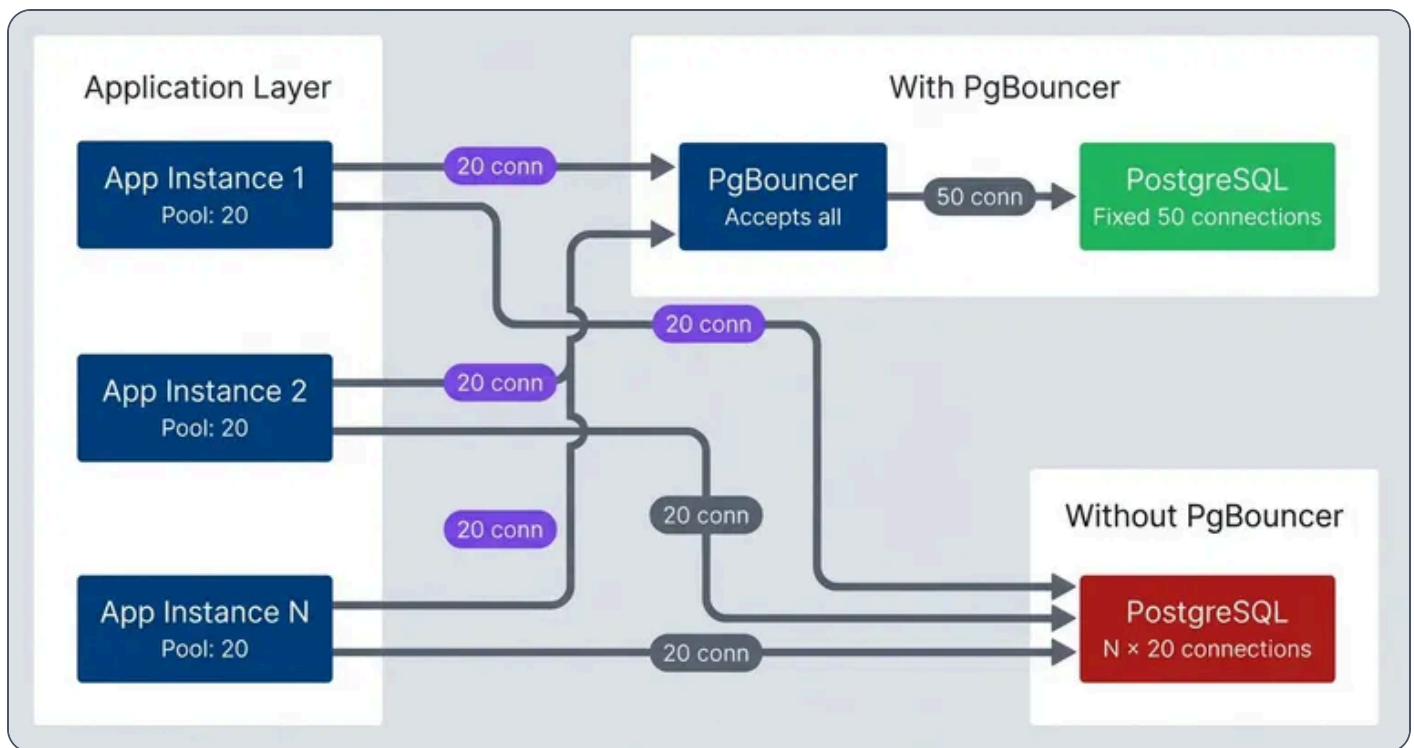
Connection returned after each transaction completes. High multiplexing, works for most web applications. Prepared statements work only with server-side preparation.



Statement mode

Connection returned after each statement. Maximum multiplexing but no multi-statement transactions. Use only for simple, stateless read workloads.





PgBouncer connection multiplexing.

PgBouncer Configuration

PgBouncer (PostgreSQL Connection Pooler) configuration lives in `pgbouncer.ini`. The critical settings control connection limits, pooling behavior, and timeouts.

`pgbouncer.ini`

```

1  [databases]
2  myapp = host=postgres.internal port=5432 dbname=myapp
3
4  [pgbouncer]
5  ; Connection limits
6  max_client_conn = 1000      ; Max client connections PgBouncer accepts
7  default_pool_size = 20     ; Server connections per pool (user+db combo)
8  min_pool_size = 5          ; Keep this many connections warm
9  reserve_pool_size = 5      ; Extra connections for burst handling
10 reserve_pool_timeout = 3    ; Seconds to wait before using reserve pool
11

```



```

12 ; Pooling mode
13 pool_mode = transaction      ; session | transaction | statement
14
15 ; Timeouts
16 server_idle_timeout = 600    ; Close idle server connections after 10 min
17 query_wait_timeout = 120    ; Max seconds client waits for connection
18 server_lifetime = 3600      ; Recycle connections after 1 hour
19
20 ; Health checks
21 server_check_delay = 30
22 server_check_query = SELECT 1
23
24 ; Admin interface
25 listen_addr = 0.0.0.0
26 listen_port = 6432
27 admin_users = pgbouncer_admin

```

PgBouncer (PostgreSQL Connection Pooler) configuration for a typical web application.

➤ Pool sizing

Follows a simple formula: $\text{default_pool_size} = \text{PostgreSQL max_connections} / \text{number_of_pools}$. If PostgreSQL allows 100 connections and you have 5 user + database combinations (pools), each pool gets 20 connections. The `reserve_pool` handles burst traffic – set it to 10-25% of `default_pool_size`.

➤ max_client_conn

Determines how many application connections PgBouncer accepts. This can be much higher than PostgreSQL's `max_connections` because PgBouncer queues requests when all server connections are busy. Set this based on peak concurrent application connections with headroom for spikes.

Scenario	max_client_conn	default_pool_size	PostgreSQL max_connections
Small web app (5 instances)	200	15	100
Medium service (20 instances)	500	25	200
High scale (100+ instances)	2000	50	400



PgBouncer sizing by deployment scale.

Pool Sizing

Sizing Calculations

Pool sizing isn't guesswork – it's queuing theory. Little's Law provides the foundation: $L = \lambda \times W$, where L is the average number of connections in use, λ is the request arrival rate (queries per second), and W is the average time per query.

If your application handles 100 queries per second with an average query time of 50ms, you need $100 \times 0.05 = 5$ connections on average. But Little's Law uses averages, and averages deceive. Query times vary. Traffic spikes. A pool sized for average load will saturate during peaks.

This is why the practical formula substitutes P99 query time for average: you want enough connections to handle 99% of queries without queueing, not just the typical case. Then double for variance and add burst capacity.

The practical formula for application pool sizing:

$$\text{pool_size} = \text{ceil}(\text{peak_gps} \times \text{p99_query_time} \times \text{burst_factor})$$

For an application instance handling 50 queries per second at peak, with P99 query time of 100ms and a 2x burst factor: $\text{ceil}(50 \times 0.1 \times 2) = 10$ connections per instance.

#	Factor	What to Measure	Impact
1	Query duration	P99 latency, not average	Longer queries need more connections
2	Request rate	Peak QPS, not average	Higher rate needs more connections
3	Variance	Standard deviation / mean	High variance needs more headroom



#	Factor	What to Measure	Impact
4	Burst factor	Peak / baseline ratio	Typically 2-3× for web traffic

Pool sizing factors.

Common anti-patterns undermine even correct calculations:

Oversized pools:

Setting `pool_size = 100` "just in case" means 10 instances create 1000 connections. Size based on actual need.

No max lifetime:

Connections held forever accumulate memory leaks and stale state. Recycle every 30 minutes.

No idle timeout:

Idle connections waste resources and hide leaks. Timeout after 10 minutes of inactivity.

Application Pool Configuration

Application-side pools need careful configuration. The pool library handles connection reuse, but you control the parameters that determine behavior under load.

nodejs-pool-config.ts

```

1  import { Pool, PoolConfig } from 'pg'
2
3  const poolConfig: PoolConfig = {
4    host: process.env.DB_HOST,
5    port: parseInt(process.env.DB_PORT || '5432'),
6    database: process.env.DB_NAME,
7    user: process.env.DB_USER,
8    password: process.env.DB_PASSWORD,
9
10   // Pool sizing: peak_qps × p99_latency × burst_factor
11   max: 20,
12   min: 5, // Keep pool warm
13

```



```
14 // Timeouts
15 connectionTimeoutMillis: 10000, // Fail fast if pool exhausted
16 idleTimeoutMillis: 30000, // Reclaim idle connections
17 // Note: pg-pool lacks maxLifetime; implement connection recycling
18 // manually if needed (track connection age, close after 30min)
19
20 // SSL for production
21 ssl: process.env.NODE_ENV === 'production' ? {
22   rejectUnauthorized: true,
23   ca: process.env.DB_CA_CERT,
24 } : false,
25 }
26
27 const pool = new Pool(poolConfig)
28
29 // Set statement timeout on new connections
30 pool.on('connect', (client) => {
31   client.query('SET statement_timeout = 30000')
32 })
33
34 // Expose metrics for monitoring
35 function collectPoolMetrics() {
36   return {
37     total: pool.totalCount,
38     idle: pool.idleCount,
39     waiting: pool.waitingCount,
40     utilization: pool.totalCount > 0
41       ? (pool.totalCount - pool.idleCount) / pool.totalCount
42       : 0,
43   }
44 }
45
46 export { pool, collectPoolMetrics }
```

Node.js pg-pool configuration with monitoring.

For Java applications, HikariCP is the standard choice. It's fast, well-maintained, and provides excellent metrics out of the box.



HikariPoolConfig.java

```
1  import com.zaxxer.hikari.HikariConfig;
2  import com.zaxxer.hikari.HikariDataSource;
3
4  public class DatabaseConfig {
5      public static HikariDataSource createDataSource() {
6          HikariConfig config = new HikariConfig();
7
8          config.setJdbcUrl("jdbc:postgresql://localhost:5432/mydb");
9          config.setUsername(System.getenv("DB_USER"));
10         config.setPassword(System.getenv("DB_PASSWORD"));
11
12         // Pool sizing
13         config.setMaximumPoolSize(20);
14         config.setMinimumIdle(10);
15
16         // Timeouts
17         config.setConnectionTimeout(10000); // 10s to acquire connection
18         config.setIdleTimeout(600000); // 10min idle before removal
19         config.setMaxLifetime(1800000); // 30min max connection age
20
21         // Validation
22         config.setConnectionTestQuery("SELECT 1");
23         config.setKeepaliveTime(300000); // 5min keepalive
24
25         // Leak detection
26         config.setLeakDetectionThreshold(60000); // Warn if held > 1min
27
28         // Statement timeout
29         config.addDataSourceProperty("options", "-c statement_timeout=30000");
30
31         return new HikariDataSource(config);
32     }
33 }
```

HikariCP configuration for PostgreSQL.

Regardless of language or pool library, the same principles apply. These settings represent battle-tested defaults that work for most applications:



Setting	Recommended Value	Rationale
max_pool_size	10-30 per instance	Based on Little's Law calculation
min_pool_size	50% of max	Keep pool warm, reduce latency
connection_timeout	10s	Fail fast if pool exhausted
idle_timeout	10-30 min	Reclaim unused connections
max_lifetime	30 min	Prevent stale connections
leak_detection	1-5 min	Catch unreturned connections

Pool configuration guidelines.

⚠ WARNING

Never set pool `max_size` to PostgreSQL `max_connections`. Leave headroom for admin connections, monitoring, migrations, and other applications. A good rule: application pools should use at most 80% of `max_connections`.

With pools configured, the next question is: how do you know if they're working? Proper sizing is only half the battle – you need visibility into whether your pools have headroom or are approaching saturation.

Saturation Signals

Metrics to Monitor

Connection saturation doesn't announce itself with a single metric. It's a constellation of signals that, together, tell you whether your system has headroom or is approaching collapse. The goal is catching saturation early – before "too many connections" errors appear in logs.



1

Connection utilization

The primary signal. Query `pg_stat_activity` to compare current connections against `max_connections`. Alert at 70% utilization, investigate at 85%. By 95%, you're one slow query away from exhaustion.

connection-utilization.sql

```
1  SELECT
2      count(*) AS current_connections,
3      current_setting('max_connections')::int AS max_connections,
4      round(100.0 * count(*) / current_setting('max_connections')::int, 1) AS
      utilization_pct
5  FROM pg_stat_activity
6  WHERE backend_type = 'client backend';
```

Connection utilization query.

2

Pool wait time

Measures how long requests wait for an available connection from the application pool. This metric comes from your pool library (HikariCP, pg-pool), not PostgreSQL. P99 wait time above 100ms indicates the pool is undersized; above 1 second means requests are timing out.

3

Queue depth

Counts requests waiting for connections. In PgBouncer, check `cl_waiting`. In application pools, check `waitingCount` or equivalent. Any sustained queue indicates saturation – the pool can't keep up with demand.

4

Idle in transaction

Connections are the silent killers. They hold connections hostage without doing work. Query for sessions stuck in this state for more than a minute – these are usually application bugs (missing commits, unclosed transactions in error paths).

idle-in-transaction.sql

```
1  SELECT pid, username, application_name, state, query,
```



```

2    now() - state_change AS idle_duration
3  FROM pg_stat_activity
4  WHERE state = 'idle in transaction'
5    AND (now() - state_change) > interval '1 minute'
6  ORDER BY idle_duration DESC;

```

Find idle-in-transaction sessions.

Each metric has different thresholds for when to pay attention versus when to wake someone up at 3am. These thresholds are starting points – adjust based on your application’s tolerance for latency and your database’s capacity:

#	Metric	Warning Threshold	Critical Threshold	Response
1	Connection utilization	> 70%	> 85%	Add pooler or reduce pool sizes
2	Pool wait time (P99)	> 100ms	> 1s	Increase pool size or optimize queries
3	Queue depth	> 10	> 50	Pool undersized or query bottleneck
4	Idle in transaction	> 5 connections	> 20 connections	Find and fix leaking transactions
5	Connection errors	Any occurrence	Sustained errors	Immediate investigation

Saturation metric thresholds.

For PgBouncer (PostgreSQL Connection Pooler) deployments, the `SHOW POOLS` command exposes additional metrics. Watch `cl_waiting` (clients waiting for server connections), `sv_idle` (available server connections – zero means exhaustion), and `maxwait` (longest current wait time in seconds).



Monitoring Dashboard

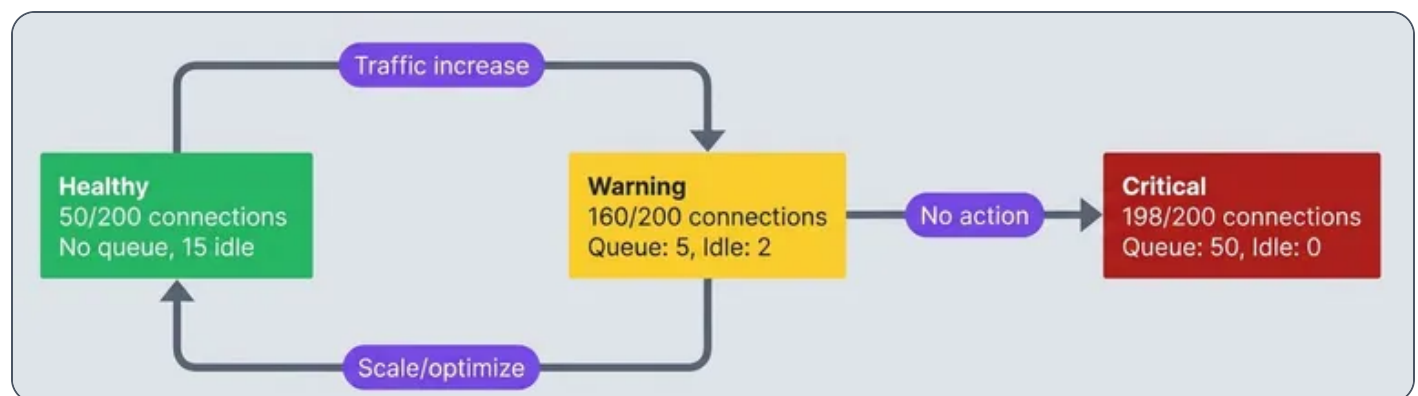
During incidents, I need quick answers: Are connections the bottleneck? Are they being recycled? Is something blocking?

The **connection snapshot query** (see Reference Queries) gives a real-time breakdown of connection states and utilization – run this first during any database performance investigation.

Connection age distribution reveals recycling problems. If you see connections older than your `max_lifetime` setting (or hours/days old when you expect minutes), you have a leak or misconfiguration.

Lock blocking analysis identifies chains where one query holds locks that others are waiting for. When connections pile up, contention is often the culprit.

The progression from healthy to critical follows a predictable pattern:



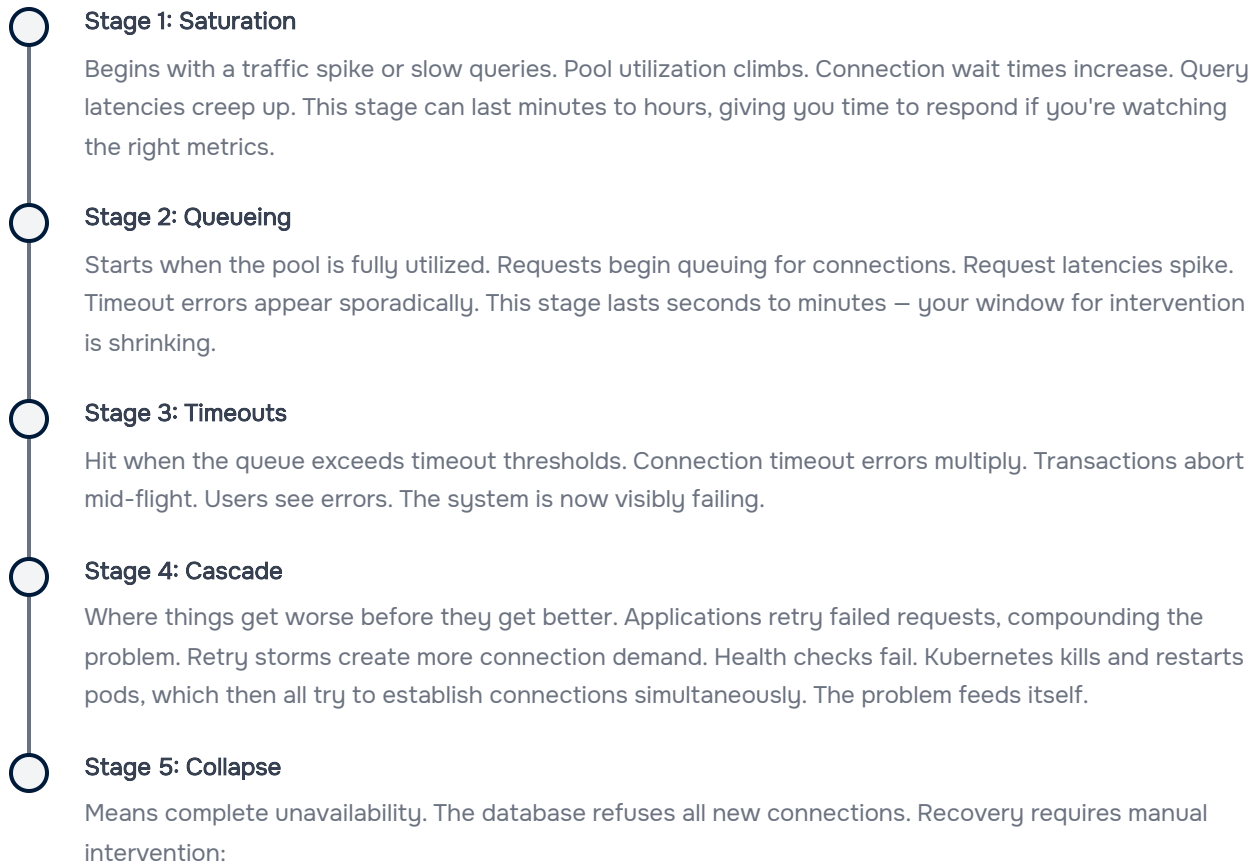
Connection saturation progression.

Failure Patterns

Connection Exhaustion Cascade

Connection exhaustion rarely happens in isolation. It cascades. Understanding the stages helps you recognize where you are in the failure progression and what interventions are still possible.





Once you reach Stage 5, the instinct is to restart everything at once. Resist it. A “thundering herd” recovery – where every application instance reconnects simultaneously – recreates the exact conditions that caused the collapse. Each instance tries to fill its connection pool, slamming the database with hundreds of concurrent connection requests before it has finished cleaning up the previous mess. The recovery playbook matters as much as the prevention strategy, and it follows a specific sequence:

1

Stop the bleeding

Scale application instances to zero or block traffic at the load balancer. This stops new connection attempts.

2

Clear stuck connections

On PostgreSQL, run `SELECT pg_terminate_backend(pid) FROM pg_stat_activity WHERE state = 'idle in transaction' AND state_change < now() - interval '5 minutes';` to kill stuck sessions.



3

Restart applications gradually

Bring up one instance, verify it connects and serves traffic, then add more. Don't start all instances simultaneously – they'll all try to fill their pools at once.

4

Monitor during recovery

Watch `pg_stat_activity` connection count as you add instances. Stop scaling if you approach 80% utilization.

Recovery gets you back to operational, but it doesn't prevent the next incident. The cascade pattern repeats because the underlying architecture lacks defensive layers – there's nothing between "working fine" and "total collapse." Building resilience means adding checkpoints where the system can absorb pressure instead of transmitting it. Each layer below addresses a different failure mode, and they compound: connection limits cap the blast radius, circuit breakers stop the bleeding, backpressure prevents the bleeding from starting, and graceful degradation keeps users served even when the database is under strain.

Connection limits

Set maximums at application pools, PgBouncer, and PostgreSQL. The principle is "fail fast at the application, not the database."



Circuit breakers

Detect saturation and return fast errors instead of queueing. Let the system recover.



Backpressure

Rate limit at the ingress. Use bounded queues. Shed load during saturation.



Graceful degradation

Disable non-critical features under stress. Serve cached responses. Return partial results.



Recovery Strategies

When saturation hits, you need patterns that prevent cascade failures. These aren't theoretical – they're the difference between a slow day and a multi-hour outage.



Circuit breakers stop the bleeding. When connection errors exceed a threshold, the circuit “opens” and immediately rejects requests instead of queuing them. This gives the database breathing room to recover. After a timeout, the circuit enters a “half-open” state, allowing a few requests through to test whether the database has recovered.

Circuit breakers for database connections live in your application code, wrapping the connection pool. They’re not implemented in PgBouncer (PostgreSQL Connection Pooler) (which has no circuit breaker concept) or at the ingress controller (which operates at HTTP level, not database protocol). The application is the right place because it understands the difference between a transient connection timeout and a sustained outage.

The essential logic: track consecutive failures, “open” the circuit after a threshold, reject requests immediately while open, then periodically test with a single request to see if the database has recovered.

circuit-breaker-usage.ts

```
1 // Circuit breaker wraps the pool, not individual queries
2 const breaker = new DatabaseCircuitBreaker({ threshold: 5, timeout: 30000 })
3 const pool = new Pool(poolConfig)
4
5 // All database access goes through the breaker
6 async function getUser(id: string) {
7   return breaker.query(pool, 'SELECT * FROM users WHERE id = $1', [id])
8 }
```

Circuit breaker usage pattern - Full implementation <<https://gist.github.com/webstackdev/b74864012c98b410435dc0aabf1f5a75>>.

Retry budgets prevent retry storms. Instead of retrying every failed request, track the ratio of retries to total requests. When retries exceed 20% of traffic, stop retrying – you’re making things worse. Wire the retry budget into your circuit breaker:

retry-with-budget.ts

```
1 const budget = new RetryBudget({ windowMs: 60000, maxRetryRatio: 0.2 })
2
3 async function queryWithRetry<T>(sql: string, params?: unknown[]): Promise<T> {
4   budget.recordRequest()
5   try {
```



```

6      return await breaker.query<T>(pool, sql, params)
7    } catch (error) {
8      if (budget.canRetry() && isRetryable(error)) {
9        budget.recordRetry()
10       return breaker.query<T>(pool, sql, params) // Single retry attempt
11     }
12     throw error
13   }
14 }

```

Integrating retry budget with circuit breaker.

Each failure scenario has a different detection method and response. The key is matching the response to the root cause:

Scenario	Detection	Response	Recovery
Slow queries	Wait time > threshold	Add query timeout	Optimize queries
Connection leak	Idle-in-txn > limit	Kill sessions	Fix application code
Traffic spike	Utilization > 80%	Shed load	Scale or add pooler
Database overload	Connection errors	Circuit break	Scale database

Failure scenarios and responses.

Conclusion

PostgreSQL connection management is a critical but often neglected aspect of application architecture. Most teams don't think about it until 3am when the database stops accepting connections.

The key insights from this article:



- ✓ **Connections are expensive.** –
Each connection costs 5-10MB of memory, consumes CPU through context switching, and competes for shared resources. The counterintuitive truth: fewer connections often means better performance. Keep `max_connections` low and use pooling.
- ✓ **Pooling decouples application scaling from database limits.** –
PgBouncer (PostgreSQL Connection Pooler) (or similar external poolers) lets you scale application instances without proportionally increasing database connections. Transaction pooling mode handles most workloads well.
- ✓ **Size pools with math, not intuition.** –
Little's Law ($L = \lambda \times W$) gives you a starting point. Account for variance with a burst factor. Anti-patterns like oversized pools ("just in case") create more problems than they solve.
- ✓ **Monitor saturation, not just errors.** –
By the time you see "too many connections" errors, you're already in a cascade. Set alerts on the utilization thresholds discussed earlier. Watch for idle-in-transaction sessions – they're connection leaks waiting to bite you.
- ✓ **Design for graceful degradation.** –
Circuit breakers, retry budgets, and backpressure turn connection exhaustion from a hard failure into a soft degradation. Build these patterns before you need them.

The goal isn't to handle unlimited connections – it's to handle connection constraints gracefully, so saturation causes slowdowns, not outages.

INFO

Connection management is about building slack into the system. A database running at 90% connection capacity has no room for variance, spikes, or slow queries. Target 50-70% utilization to leave room for reality to differ from expectations.



Reference Queries

These SQL (Structured Query Language) queries are useful for monitoring and debugging PostgreSQL connection issues. Run them against your database during incidents or as part of regular health checks.

Connection Snapshot

Real-time breakdown of connection states and utilization:

connection-snapshot.sql

```
1  SELECT
2      count(*) AS total_connections,
3      count(*) FILTER (WHERE state = 'active') AS active,
4      count(*) FILTER (WHERE state = 'idle') AS idle,
5      count(*) FILTER (WHERE state = 'idle in transaction') AS idle_in_txn,
6      current_setting('max_connections')::int AS max_conn,
7      round(100.0 * count(*) / current_setting('max_connections')::int, 1) AS
      utilization_pct
8  FROM pg_stat_activity
9  WHERE backend_type = 'client backend';
```

Connection snapshot query.

Connection Age Distribution

Identifies connections that aren't being recycled:

connection-age-distribution.sql

```
1  SELECT
2      CASE
3          WHEN age < interval '1 minute' THEN '< 1 min'
4          WHEN age < interval '10 minutes' THEN '1-10 min'
5          WHEN age < interval '1 hour' THEN '10-60 min'
6          WHEN age < interval '1 day' THEN '1-24 hours'
7          ELSE '> 1 day'
```



```

8      END AS age_bucket,
9      count(*) AS connections
10 FROM (
11     SELECT now() - backend_start AS age
12     FROM pg_stat_activity
13     WHERE backend_type = 'client backend'
14 ) ages
15 GROUP BY 1
16 ORDER BY min(age);

```

Connection age distribution query.

Lock Blocking Analysis

Finds queries blocking other queries:

blocking-analysis.sql

```

1  WITH blocked AS (
2      SELECT
3          blocked.pid AS blocked_pid,
4          blocked.query AS blocked_query,
5          blocking.pid AS blocking_pid,
6          blocking.query AS blocking_query,
7          now() - blocked.state_change AS blocked_duration
8  FROM pg_stat_activity blocked
9  JOIN pg_locks blocked_locks ON blocked.pid = blocked_locks.pid
10 JOIN pg_locks blocking_locks
11     ON blocked_locks.locktype = blocking_locks.locktype
12     AND blocked_locks.database IS NOT DISTINCT FROM blocking_locks.database
13     AND blocked_locks.relation IS NOT DISTINCT FROM blocking_locks.relation
14     AND blocked_locks.pid != blocking_locks.pid
15 JOIN pg_stat_activity blocking ON blocking_locks.pid = blocking.pid
16 WHERE NOT blocked_locks.granted
17 )
18 SELECT * FROM blocked
19 WHERE blocked_duration > interval '10 seconds'
20 ORDER BY blocked_duration DESC;

```



Lock blocking analysis query.

Kill Stuck Sessions

Emergency cleanup of idle-in-transaction sessions (use with caution):

kill-stuck-sessions.sql

```
1  SELECT pg_terminate_backend(pid)
2  FROM pg_stat_activity
3  WHERE state = 'idle in transaction'
4  AND state_change < now() - interval '5 minutes';
```

Kill idle-in-transaction sessions older than 5 minutes.

-
- ① Effective spindle count represents the hardware's capacity for concurrent I/O requests. A traditional HDD has one spindle and can usually only handle one I/O request at a time. For modern SSD-based storage, this metric is often considered 0 or ignored in the original formula. Because SSDs handle many parallel I/O requests, some modern benchmarks suggest using I/O queue depth as a more relevant replacement. ↩

Copyright © 2023 Webstack Builders, Inc.

The text, diagrams, and images in this work are licensed under CC BY-NC 4.0

All code samples in this article are licensed under the MIT License. Feel free to use, modify, and distribute them in any project.

<https://www.webstackbuilders.com/articles/postgresql-connection-pooling-saturation-sizing>

